Collaborative Research in APA 7th Edition

Alexander Hoem, Michael Huse, Noah Øfsti, and Niklas Goldhan

Department of Computer Science

Oslo Metropolitan University

ACIT4610: Evolutionary Artificial Intelligence and Robotics

Dr. Jane Vance

November 15, 2026

Collaborative Research in APA 7th Edition

Alexander

Collaborative academic writing requires coherent citation practices across all contributing authors. In APA 7, parenthetical citations place the authors and year inside parentheses (Smith & Doe, 2023). When citing specific pages or sections, provide the page locator directly in the bracket argument (Smith & Doe, 2023, p. 46). Multiple sources supporting a single claim can be grouped within one command and are automatically alphabetized (Eiben & Smith, 2015; Smith & Doe, 2023).

Citation Strategies in APA 7

Narrative citations integrate the author directly into the sentence flow, with only the publication year enclosed in parentheses. As Miller (2021) demonstrated, structured documentation significantly lowers technical barriers for new collaborators. When referencing foundational models, early seminal papers can be cited narratively alongside algorithmic descriptions (Kennedy & Eberhart, 1995).

Git Workflow and Conflict Prevention

Git tracks changes line by line rather than word by word. Writing in a semantic line break style—placing exactly one sentence per line—isolates modifications to specific diffs. This strategy enables teammates to edit adjacent sentences without generating false merge conflicts.

Michael

Clear visual hierarchy and structured data representation enhance the readability of empirical reports. In APA 7, introduce technical terms or key terminology with *italics* upon first mention rather than boldface. Reserve **bold text** primarily for table titles, section headings, and specific mathematical vectors.

Text Formatting and Lists

Unordered lists are appropriate when the sequence of elements has no inherent chronological or ranked order. In APA 7, list items that form complete sentences must begin with a capital letter and conclude with appropriate punctuation:

- Selection pressures govern the reproductive probability of candidate solutions.
- Crossover operators facilitate the recombination of genetic material between parents.
- Mutation mechanisms maintain population diversity across evolutionary epochs.

APA 7 Compliant Tables

Tables in APA 7 omit vertical borders and rely strictly on horizontal rules at the top, header baseline, and footer. The table number appears in bold, followed by a double-spaced title in italicized title case. Table notes positioned immediately below the table provide abbreviations and explanatory context.

Table 1

Baseline Hyperparameters for Algorithmic Comparison

Algorithm	Selection Operator	Population Size	Mutation Rate
GA	Tournament ($k = 3$)	100	.015
ES	(μ, λ) Selection	60	.050
PSO	Global Best Topology	80	—

Note. GA = Genetic Algorithm; ES = Evolution Strategy; PSO = Particle Swarm Optimization. Dashes indicate parameters that do not apply to the specific algorithm. Floating rates below 1.0 omit leading zeros in accordance with APA 7 standards.

Noah

Empirical evaluations in evolutionary computing require rigorous statistical reporting and transparent procedural documentation. Sequential experimental protocols

should be structured using numbered enumeration to communicate order of execution clearly.

Numbered Sequences and Procedures

When documenting iterative experimental benchmarks, numbered lists establish the exact procedural sequence:

1. Random seeds and population vectors were initialized uniformly across sample spaces.
2. Fitness evaluations ran over 50 independent runs for each algorithmic variant.
3. Non-parametric statistical tests were executed to evaluate convergence speed.

Statistical and Mathematical Notation

APA 7 mandates specific typographic rules for statistical reporting. All single-letter statistical symbols and variables—such as p, t, F, M, SD, and d—must appear in italics. Numbers, parentheses, and Greek letters like α or η_p^2 remain upright. Furthermore, numbers bounded between 0 and 1 that cannot theoretically exceed 1.0 (including p-values, correlation coefficients r, and proportions) omit the leading zero.

The evolutionary strategy demonstrated superior fitness over the baseline, $M = 94.20$, $SD = 3.15$, compared to the control group, $M = 88.50$, $SD = 4.22$. An independent samples t-test revealed a statistically significant difference, $t(48) = 5.41$, $p < .001$, $d = 1.53$, with a 95% confidence interval around the difference, 95% CI $[3.58, 7.82]$. Similarly, analysis of variance confirmed main effects across benchmark landscapes, $F(2, 72) = 8.64$, $p = .002$, $\eta_p^2 = .19$. When accompanied by specific numerals, format percentages using the symbol, such as 95% confidence and 12% variance, but use the word “percentage” in abstract text.

Part A: Travelling Salesman Problem (TSP)

Bees Algorithm for TSP

The Bees Algorithm (BA) is an algorithm that is inspired by nature that mimics the foraging behavior of honey bees to solve optimization problems. For the Travelling Salesman Problem (TSP), it balances the global exploration across the search space (scout bees) with a more focused local exploitation around the most promising routes (recruited forager bees).

Problem Formulation

The Travelling Salesman Problem requires finding the shortest complete round trip that visits n distinct cities exactly once and returns to the starting city.

- **Solution Representation:** A single route is encoded as a permutation of city indices:

$$\pi = (\pi_1, \pi_2, \ldots, \pi_n) \tag{1}$$

  City 1 is fixed at index 0 ($\pi_1 = 1$) to make sure the beginning and end are always in the same spot. It is only appended at the end when the script calculates the return journey or displaying the route.

- **Objective Function:** The total route length $L(\pi)$ sums all consecutive legs plus the return leg to City 1:

$$L(\pi) = \sum_{i=1}^{n-1} d_{\pi_i, \pi_{i+1}} + d_{\pi_n, \pi_1} \tag{2}$$

- **Percentage Gap:** Measures the relative solution quality against the known TSPLIB optimum (L^*):

$$\text{Gap}(\%) = 100 \times \frac{L_{\text{BA}} - L^*}{L^*} \tag{3}$$

Data and Distance Calculation

Benchmark Instances

The experiments use six standard 2D symmetric TSP instances from the TSPLIB library:

Table 2

TSPLIB 2D Symmetric Benchmark Instances

Instance	Cities (n)	Edge Weight Type	Known Optimum (L^*)
`eil51`	51	`EUC_2D`	426
`berlin52`	52	`EUC_2D`	7,542
`st70`	70	`EUC_2D`	675
`eil76`	76	`EUC_2D`	538
`kroA100`	100	`EUC_2D`	21,282
`ch150`	150	`EUC_2D`	6,528

Note. All instances use 2D Euclidean rounded edge weights. Known optimum values (L^*) are taken from TSPLIB reference documentation.

Distance Calculation (`calc_distance`)

Following the TSPLIB `EUC_2D` standard, distances between coordinates (a_i, b_i) and (a_j, b_j) are rounded down to the nearest integer:

$$d_{ij} = \left\lfloor \sqrt{(a_i - a_j)^2 + (b_i - b_j)^2} + 0.5 \right\rfloor \tag{4}$$

Each edge is rounded individually before summing. A precomputed $n \times n$ distance matrix is build at the start (`create_distance_matrix`) for fast distance lookups during the run.

Validation Checks (`validate_dataset_metadata`)

Before running the algorithm, the parser validates the input files:

- Checks that `TYPE == 'TSP'` and `EDGE_WEIGHT_TYPE == 'EUC_2D'`.

- Verifies that `DIMENSION` matches the number of parsed coordinate rows.
- Checks that all city identifiers are unique.
- Confirms that every generated route contains each city exactly once.

Neighborhood Operators

Both operators leave City 1 fixed at index 0 and sample from the remaining positions $\{1, \ldots, n-1\}$, guaranteeing that all generated neighbors remain valid permutations without needing repair.

Swap Operator (`swap_operator`)

Randomly selects two different indices i and j from $\{1, \ldots, n-1\}$ and exchanges their cities:

- `new_trip[i] = trip[j]`, `new_trip[j] = trip[i]`.
- Exchanges the positions of two cities.
- Breaks up to 4 existing edges in the route when i and j are non-adjacent.

Segment Reversal Operator (`reverse_operator`)

Randomly selects two different indices $i < j$ from $\{1, \ldots, n-1\}$ and reverses the entire segment between them:

- `new_trip[i : j + 1] = reversed(trip[i : j + 1])`.
- Equivalent to a classical 2-opt inversion move.
- Breaks exactly 2 edges and reconnects them while preserving all internal pairwise connections along the reversed segment.

Bees Algorithm Main Loop

The algorithm runs until reaching exactly 30,000 objective evaluations per run.

- **Initialization (`init_population`):** Calls `generate_initial_trip()` to create $n = 20$ random scout routes with City 1 fixed at index 0. Calls `evaluator.evaluate()` on each scout to compute route length and updates the best-so-far record. Sorts the initial population in ascending order of route length.
- **Elite Sites Local Search (`search_patch`):** Selects the top $e = 2$ elite routes from the population. For each elite site, recruits $n_e = 20$ forager bees. Each forager mutates the route using the chosen operator and evaluates its length. Keeps only the single best route found within that patch (greedy selection).
- **Selected Best Sites Local Search (`search_patch`):** Selects the next $m - e = 4$ best routes ($m = 6$ total selected best sites). For each site, recruits $n_s = 8$ forager bees, generates mutated neighbors, and keeps only the patch winner.
- **Global Scouting:** For the remaining $n - m = 14$ slots, generates fresh random scout routes using `generate_initial_trip()`, evaluates each, and adds them to the new population to maintain diversity.
- **Population Update and Termination:** Replaces the population with the 20 new bees (2 elite winners, 4 non-elite winners, 14 random scouts) and sorts by route length. Each full iteration evaluates $(2 \times 20) + (4 \times 8) + 14 = 86$ possible solutions. Repeats until `evaluator.is_budget_exhausted()` reaches 30,000 evaluations. Returns `best_trip`, `best_length`, and the evaluation history.

Experimental Setup

- **Runs:** 10 differnet random seeds per instance and operator ($S = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512]$), totaling 120 TSP runs (6 instances $\times$ 2 operators $\times$ 10 seeds).

- **Evaluation Budget:** Exactly 30,000 objective evaluations per run.
- **Parameters:** Population size $n = 20$, selected best sites $m = 6$, elite sites $e = 2$, elite foragers $n_e = 20$, non-elite selected foragers $n_s = 8$.
- **Metrics:** Best route length, mean route length, standard deviation, mean percentage gap, and mean runtime in seconds.

Results

Summary Statistics

Table 3

Performance Comparison of Swap and Segment Reversal Operators on TSPLIB Instances

Instance	Cities	Operator	Optimum (L^*)	Best	Mean	*SD*	Mean Gap (%)	Mean Runtime (s)
`eil51`	51	Swap	426	535	567.6	20.16	33.24%	0.08
	51	**Reversal**	426	**444**	**457.3**	**9.97**	**7.35%**	0.09
`berlin52`	52	Swap	7,542	9,455	10,356.8	501.44	37.32%	0.11
	52	**Reversal**	7,542	**7,874**	**8,140.0**	**208.44**	**7.93%**	0.12
`st70`	70	Swap	675	1,026	1,156.0	64.53	71.26%	0.13
	70	**Reversal**	675	**710**	**761.9**	**21.09**	**12.87%**	0.14
`eil76`	76	Swap	538	755	823.8	37.39	53.12%	0.14
	76	**Reversal**	538	**599**	**616.7**	**13.00**	**14.63%**	0.14
`kroA100`	100	Swap	21,282	48,629	50,602.9	1,143.43	137.77%	0.19
	100	**Reversal**	21,282	**29,047**	**30,208.7**	**1,005.21**	**41.94%**	0.21
`ch150`	150	Swap	6,528	15,920	18,565.4	1,073.31	184.40%	0.27
	150	**Reversal**	6,528	**11,628**	**12,644.0**	**453.75**	**93.69%**	0.28

Note. All metrics are averaged over 10 independent runs (seeds $1, 2, 4, 8, 16, 32, 64, 128, 256, 512$) with 30,000 objective evaluations per run. Best results for each problem instance appear in bold. L^* denotes the known global optimum from TSPLIB.

Convergence Plots

The convergence curves compare evaluations (0 to 30,000) against mean best-so-far route length across 10 runs for `berlin52` and `ch150`:

- **Figure 1 (`berlin52`):** Both operators initialize from random routes with mean length ≈30,170. Reversal drops rapidly to ≈13,589 by 3,000 evaluations and reaches ≈9,183 by 10,000 evaluations, continuing the drop in length to a final mean of 8,140.0 (near the optimum $L^* = 7{,}542$). Swap experiences early stagnation, flattening to ≈12,023 by 10,000 evaluations and terminating with a sub-optimal mean of 10,356.8.
- **Figure 2 (`ch150`):** On the 150-city problem, both operators start around ≈53,277. Reversal reaches ≈21,699 by 10,000 evaluations and finishes at 12,644.0, while the Swap operator stagnates early and only reaches 18,565.4.

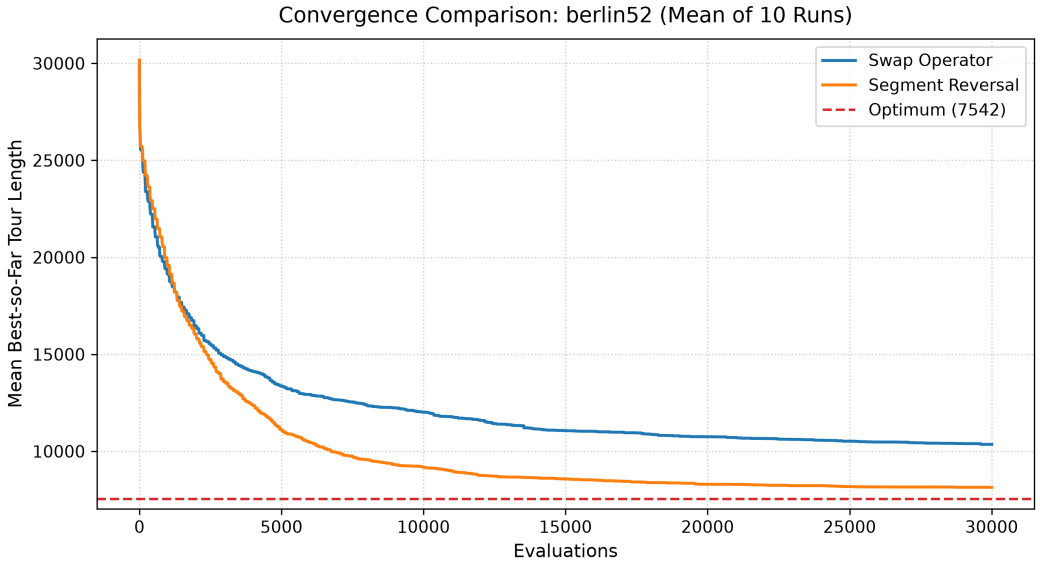


Figure 1

Mean Best-So-Far Route Length Convergence Across 10 Runs on `berlin52`

Shortest Berlin52 Route

Across all 20 runs on `berlin52`, the single shortest route found was:

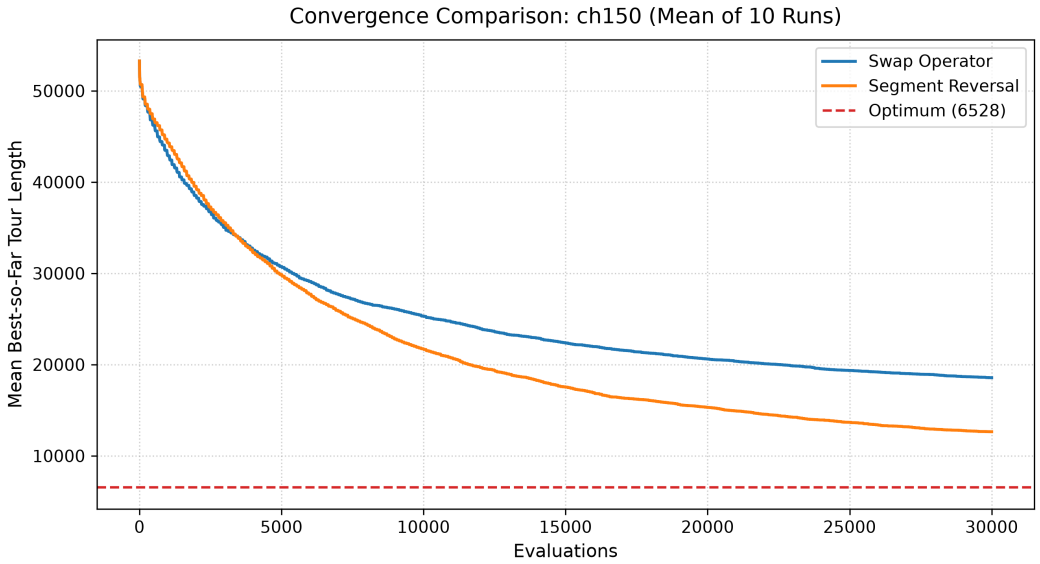


Figure 2

Mean Best-So-Far Route Length Convergence Across 10 Runs on `ch150`

- **Neighborhood:** Segment Reversal
- **Seed:** 8
- **Route Length:** 7,874
- **Optimum:** 7,542
- **Gap:** 4.40%
- **Route Visualization:** Plotted in Figure 3 with starting City 1 marked as the depot and the return leg explicitly drawn. The route follows a smooth perimeter without any self-intersecting loops.

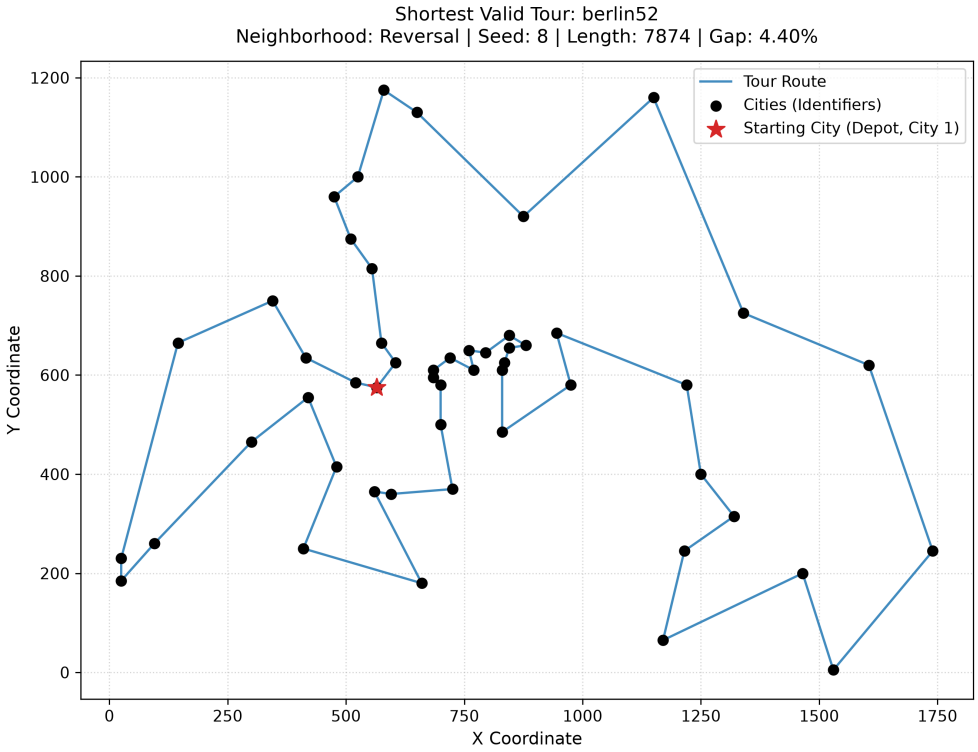


Figure 3

Shortest Route Discovered on `berlin52` *(*$L = 7{,}874$*,* $Gap = 4.40\%$*, Reversal Operator, Seed 8)*

Discussion and Analysis

Why Segment Reversal Outperforms Swap

- **Edge Preserving vs. Edge Breaking:** The route quality depends on city adjacencies (edges). Swapping two non-consecutive cities breaks up to 4 edges and replaces them with 4 random edges, possibly destroying good partial paths. Segment reversal breaks only 2 edges and reconnects them while preserving all internal city connections in reverse order.

- **Eliminating Crossings:** In Euclidean space, optimal routes never cross themselves. Reversing a segment between two crossing edges uncrosses them (standard 2-opt property). Swap moves cannot uncross edges without changing other connections.

Convergence and Stagnation

- **Early Improvement:** Both operators achieve fast improvements in the first 2,000–5,000 evaluations by eliminating large random jumps between cities.
- **Premature Stagnation in Swap:** Swap flattens early (after ≈8,000–10,000 evaluations) because single swaps cannot untangle complex route entanglements.
- **Continuous Improvements in Reversal:** Reversal continues making steady improvements throughout the full 30,000 evaluations.

Effect of Problem Scale

- For smaller instances (`eil51`, `berlin52`), Reversal reaches mean gaps below 8%.
- As the problem size grows to 100 and 150 cities, the permutation space grows factorially ($(n-1)!/2$).
- Under the fixed 30,000-evaluation budget, the fraction of the search space sampled becomes much smaller, causing mean gaps to increase to 41.94% on `kroA100` and 93.69% on `ch150`.

Consistency Across Runs

- Reversal produces consistently lower standard deviations across all 10 seeds (e.g., $SD = 9.97$ vs. 20.16 on `eil51`; $SD = 208.44$ vs. 501.44 on `berlin52`).
- The combination of elite site exploitation and continuous random scouting allows the algorithm to converge reliably regardless of the starting seed.

References

Eiben, A. E., & Smith, J. E. (2015). *Introduction to evolutionary computing* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-44874-8

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*, *4*, 1942–1948. https://doi.org/10.1109/ICNN.1995.488968

Miller, D. (2021). *The LaTeX starter manual*. Open Academic Press.

Smith, J., & Doe, J. (2023). Understanding collaborative writing workflows. *Journal of Academic Productivity*, *14*(2), 45–58. https://doi.org/10.1000/182